

Code-Layout, Readability and Reusability

Date	15 March 2024
Team ID	XXXXXX
Project Name	Human Development Index (HDI) Predictor
Maximum Marks	5 Marks

Code-Layout, Readability and Reusability:

This document evaluates the quality of the codebase in terms of its structure, readability, and reusability. Well-organized code with proper naming conventions, comments, and modular design ensures that the project is maintainable and can be extended or reused in future development.

Code Layout Checklist:

S.No	Code Quality Parameter	Description	Followed (Yes / No / Partial)	Remarks
1	Consistent Indentation	Uniform spacing/tabs used throughout The code	Yes	Python (PEP 8) and React code use consistent indentation throughout the project.
2	Proper File Structure	Files and folders are logically organized	Yes	Project is organized into backend, frontend, dataset, models, and screenshots folders.
3	Meaningful Variable Names	Variables reflect their purpose clearly	Yes	Variables such as life_expectancy, expected_schooling, predicted_hdi clearly describe their purpose.
4	Function / Method Names	Functions are descriptively named	Yes	Functions like predict_hdi(), train_model(), and API handlers use descriptive names.
5	Code Comments	Inline and block comments explain logic	Yes	Important logic and modules include comments explaining their functionality.
6	Modular Design	Code is split into reusable functions/modules	Yes	Backend, frontend, preprocessing, model training, and prediction logic are separated into reusable modules.
7	No Redundant Code	Duplicate or unused code is removed	Yes	Duplicate code removed and reusable utility functions are used where appropriate.
8	Error Handling	Exceptions and errors are handled gracefully	Yes	Flask API validates input, handles invalid requests, and

				returns proper HTTP error responses.
--	--	--	--	--------------------------------------

Reusable Components / Modules:

S.No	Component / Module Name	Language / Technology	Where Reused	Reusability Level (High / Medium / Low)
1	Prediction API (app.py)	Python/Flask	Prediction endpoint	High

2	Model Training (train_model.py)	Python / Scikit-learn	Model development	High
3	Prediction Engine (predict.py)	Python	Backend inference	High
4	React Components	React + Tailwind CSS	Dashboard UI	High
5	API Service (api.js)	Axios	Frontend-Backend communication	High

Overall Code Quality Assessment:

Aspect	Rating (1-5)	Comments
Code Layout & Structure	5/5	Well-organized folder structure with separate backend, frontend, dataset, and models.
Readability	5/5	Descriptive naming conventions and consistent formatting improve readability.
Reusability	5/5	Modular architecture allows components and functions to be reused easily.
Documentation / Comments	4.5/5	Core modules are documented; additional inline comments can be added in future updates.
Overall Score	4.9/5	Professional-quality codebase following good software engineering practices.